

ShopEz - Project Documentation

1. Introduction

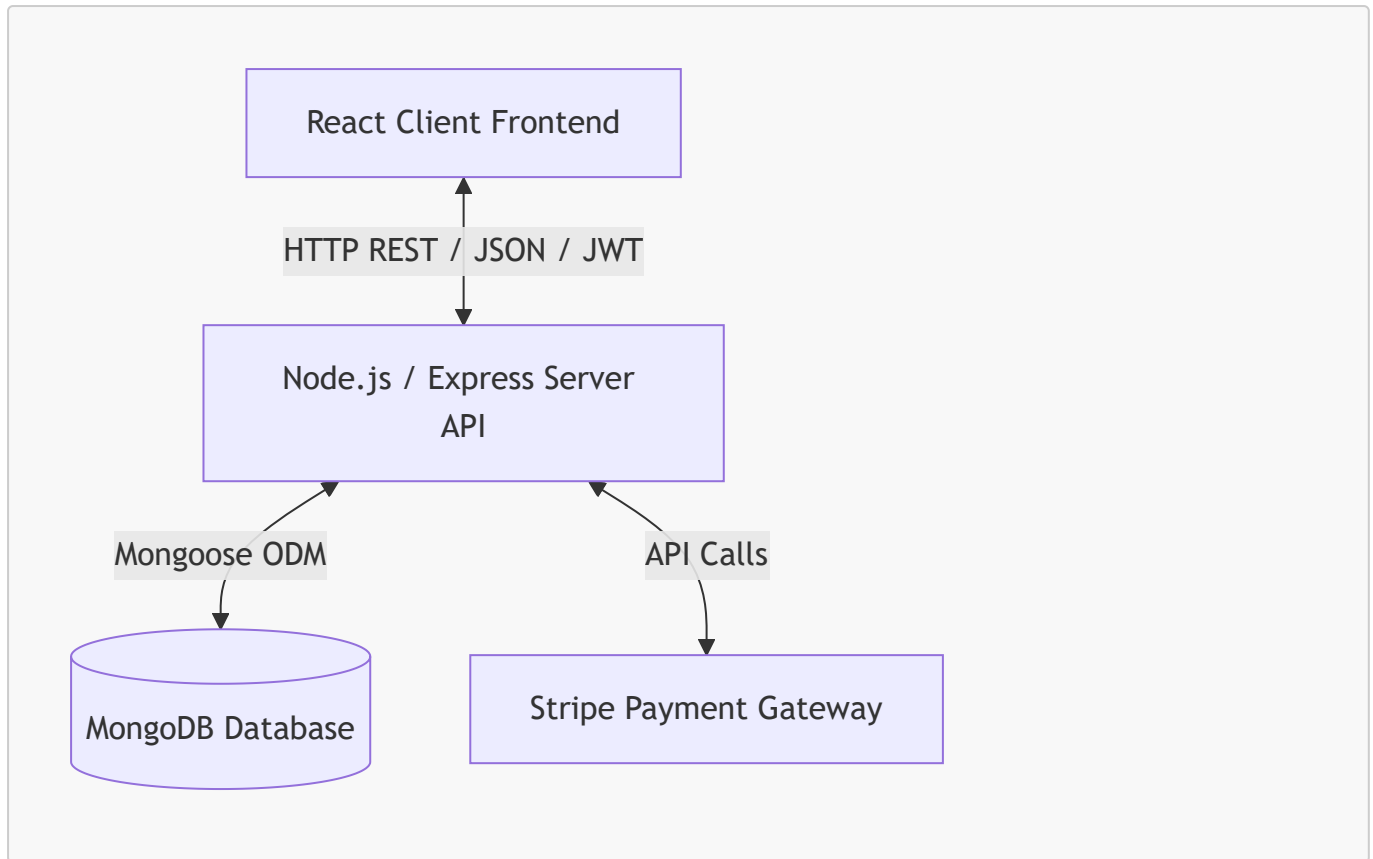
- **Project Title:** ShopEz - Full-Stack MERN E-Commerce Platform
 - **Team Members:**
 - **Full-Stack Developer:** Responsible for React frontend, Express.js backend, and MongoDB database design.
-

2. Project Overview

- **Purpose:** ShopEz is a responsive, feature-rich e-commerce web application built using the MERN stack. It aims to bridge the gap between shoppers and retailers by providing a fast, secure, and user-friendly platform for browsing products, managing a shopping cart, executing secure checkout transactions, and tracking orders.
 - **Features:**
 - **User Authentication:** Secure signup and login with JWT and hashed passwords.
 - **Product Catalog:** Dynamic list of products with search, categorization, and sorting capabilities.
 - **Shopping Cart:** Real-time cart updates, quantity changes, and price calculations.
 - **Secure Checkout:** Integration with Stripe Payment Gateway for test/live transactions.
 - **Order History & Tracking:** Detailed record of past orders and checkout status.
 - **Admin Dashboard:** Panel for administrators to manage products, categories, users, and review orders.
-

3. Architecture

ShopEz uses the modern **MERN (MongoDB, Express, React, Node.js)** architecture:



- **Frontend (React):**
 - Built as a Single Page Application (SPA).
 - Uses Redux Toolkit for centralized state management (auth status, cart items).
 - Uses React Router DOM for routing.
 - Tailwind CSS for responsive design.
- **Backend (Node.js & Express.js):**
 - RESTful API architecture.
 - Middlewares for JWT verification, input validation, and global error handling.
- **Database (MongoDB):**
 - Document-based storage utilizing Mongoose ODM.
 - Schema relations between Users, Products, and Orders.

4. Setup Instructions

Prerequisites

- **Node.js** (v16.x or higher)
- **npm** (v7.x or higher)
- **MongoDB Atlas** account or local MongoDB instance

Installation

1. Clone the repository:

```
git clone https://github.com/example/shopez.git
```

```
cd shopez
```

2. Install Frontend Dependencies:

```
cd client  
npm install
```

3. Install Backend Dependencies:

```
cd ../server  
npm install
```

4. Environment Configuration: Create a .env file in the server directory:

```
PORT=5000  
MONGO_URI=mongodb+srv://<username>:<password>@cluster.mongodb.net/shopez  
JWT_SECRET=your_jwt_secret_key_here  
STRIPE_SECRET_KEY=your_stripe_secret_key_here
```

Create a .env file in the client directory:

```
REACT_APP_API_URL=http://localhost:5000/api  
REACT_APP_STRIPE_PUBLISHABLE_KEY=your_stripe_publishable_key_here
```

5. Folder Structure

```
shopez/  
├── client/                                # React Frontend  
│   ├── public/                          # Static files  
│   └── src/  
│       ├── components/                  # Reusable UI elements (Navbar, ProductCard, etc.)  
│       ├── pages/                       # Page views (Home, ProductDetails, Cart, Checkout,  
Dashboard)  
│       ├── redux/                       # Redux slices (authSlice, cartSlice)  
│       ├── utils/                       # API helpers  
│       ├── App.js                       # App layout and routing  
│       └── index.js                     # React entrypoint  
└── server/                              # Node.js / Express Backend  
    ├── config/                          # Database connection details  
    ├── controllers/                     # Request handlers (auth, product, order controllers)  
    ├── middleware/                      # Auth verification, error handling  
    └── models/                          # Mongoose schemas (User, Product, Order)
```

```
| routes/           # API routes
| .env              # Server environment configurations
| server.js         # Server entrypoint
```

6. Running the Application

Frontend Development Server

Navigate to the **client** directory and start the server:

```
cd client
npm start
```

App will run locally at **<http://localhost:3000>**.

Backend Server

Navigate to the **server** directory and start the server:

```
cd server
npm start
```

API will run locally at **<http://localhost:5000>**.

7. API Documentation

Method	Endpoint	Description	Auth Required
POST	/api/auth/signup	Register a new user	No
POST	/api/auth/login	Login user and return token	No
GET	/api/products	Retrieve all products	No
GET	/api/products/:id	Retrieve product details	No
POST	/api/products	Add a new product (Admin only)	Yes (Admin)
POST	/api/orders	Create a new checkout session / order	Yes
GET	/api/orders/my-orders	Fetch logged-in user's order history	Yes

Example Request/Response: User Login

- **Endpoint:** **POST** **[/api/auth/login](#)**
- **Request Body:**

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

- **Response (200 OK):**

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "60d5ec4934d43f0015a1f28b",
    "name": "Jane Doe",
    "email": "user@example.com",
    "isAdmin": false
  }
}
```

8. Authentication

ShopEz utilizes a token-based authentication system:

1. **Token Generation:** Upon successful login or signup, the backend signs a JSON Web Token (JWT) with the user ID and role (admin/user) using `JWT_SECRET`.
2. **Token Transmission:** The token is sent to the client and stored securely in `localStorage` or state.
3. **Protected Routes (Frontend):** React Router guards check for the presence of the JWT token to permit entry to pages like `Checkout` and `Dashboard`.
4. **Authorized Requests (Backend):** An `authMiddleware` intercepts request headers, parses the `Authorization: Bearer <token>`, and verifies it. If valid, the user payload is attached to the request object (`req.user`).

9. User Interface

The UI is styled using a modern Tailwind CSS theme:

- **Home/Landing Page:** Hero banner, featured categories, and latest product grid.
- **Product Details Page:** Image gallery, pricing, review summary, and "Add to Cart" interactions.
- **Cart Page:** Summary table of products, quantity control adjustments, subtotal, and checkout CTA.
- **Checkout Flow:** Billing address inputs and an embedded Stripe Card Element frame.

10. Testing

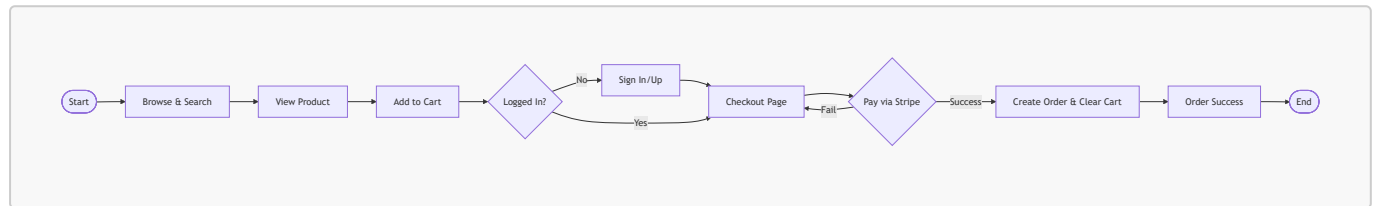
- **Backend API Testing:** Unit and integration testing conducted via **Jest** and **Supertest** targeting the controller functions and routes.

- **Manual API Testing:** Request collections run through **Postman** to verify HTTP response codes and JWT token handling.
 - **Frontend Testing:** Responsive display and cart state changes verified using **React Testing Library**.
-

11. Flowcharts

Complete User Purchase & Order Lifecycle Flowchart

The following diagram illustrates the detailed user checkout journey from browsing to payment completion and order recording:



12. Known Issues

- **Image Uploads:** Product image uploads currently rely on absolute URL strings. Local disk uploads are not persistently stored across deployment server instances.
 - **Session Expiry:** User session does not auto-logout dynamically upon JWT token expiration; users see API errors (401 Unauthorized) and must manually re-login.
-

13. Future Enhancements

- **Cloudinary Integration:** Fully migrate to Cloudinary for cloud-based media storage and image resizing.
- **Live Search Autocomplete:** Introduce search suggestions as users type query keywords.
- **Push Notifications:** Real-time notifications for price drops and delivery status tracking.